Bölüm 10: Özyineleme

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week5_Recursion
7 Kasım 2026

İçindekiler

Bölüm 10 · Özyineleme 3

Bölümün amacı 3

Konu anlatımı 3

Kendini çağıran fonksiyon (Slayt 1–5) 3

Özyinelemeyle problem çözmek (Slayt 6–7) 4

Faktöriyel (Slayt 8–11) 4

Doğrudan ve dolaylı özyineleme (Slayt 12) 4

Liste aralığını toplamak (Slayt 13–14) 5

Fibonacci ve EBOB (Slayt 15–16) 5

Hanoi kuleleri (Slayt 17–21) 6

Özyineleme mi, döngü mü? (Slayt 22–23) 6

Sık yapılan hatalar 7

Kısa tekrar soruları 7

Cevaplar 8

Bölüm 10 · Özyineleme

Bölümün amacı

Bu bölüm kendini çağıran fonksiyonlarla problem çözmeyi öğretir. Sonunda şunları yapabilmelisin:

- Özyinelemeli fonksiyonun ve özyineleme derinliğinin ne olduğunu açıklamak
- Bir problemi **temel durum** ve **özyinelemeli durum** olarak ayırmak
- Faktöriyel, liste aralığı toplamı, Fibonacci, EBOB ve Hanoi kuleleri algoritmalarını yazmak
- Doğrudan ve dolaylı özyinelemeyi ayırt etmek; özyineleme ile döngüyü karşılaştırmak

Konu anlatımı

Kendini çağıran fonksiyon (Slayt 1–5)

Özyinelemeli (recursive) fonksiyon kendini çağırır. Kaç kez tekrar edeceğini kontrol eden bir yolu olmalıdır; bu genellikle "ne zaman değer döndüreceğim, ne zaman kendimi çağıracağım" kararını veren bir `if-else`'tir. Fonksiyonun kendini kaç kez çağırdığına **özyineleme derinliği** denir. Bir çağrı bitince kontrol, o çağrının hemen sonrasına döner.

PYTHON

```python
def message(times):
    if times > 0:
        print('This is a recursive function.', times)
        message(times - 1)
    print('dönüş:', times)

message(3)
```

ÇIKTI

```
This is a recursive function. 3
This is a recursive function. 2
This is a recursive function. 1
dönüş: 0
dönüş: 1
dönüş: 2
dönüş: 3
```

Çıktıda "dönüş" satırlarının **ters sırada** gelmesi, her çağrının bitince kendisini çağıranın kaldığı yere döndüğünü gösterir.

Özyinelemeyle problem çözmek (Slayt 6–7)

Özyineleme güçlü ama **zorunlu değildir**: özyinelemeyle çözülen her problem döngüyle de çözülebilir. Her çağrının ek yükü olduğu için özyinelemeli algoritmalar genellikle **daha az verimlidir**. Genel kalıp:

- **Temel durum (base case):** problem özyineleme olmadan hemen çözülebiliyorsa çöz ve döndür.
- **Özyinelemeli durum:** değilse problemi aynı yapıda **daha küçük** bir probleme indirge ve fonksiyonu tekrar çağır.

Faktöriyel (Slayt 8–11)

n = 0 için n! = 1 (temel durum); n > 0 için n! = n × (n − 1)! (özyinelemeli durum). Her çağrıda parametre bir azalır ve sonunda temel duruma ulaşılır.

PYTHON

```python
def factorial(num):
    if num == 0:
        return 1
    else:
        return num * factorial(num - 1)

print(factorial(4), factorial(0), factorial(10))
```

ÇIKTI

```
24 1 3628800
```

`factorial(4)` = 4 × factorial(3) = 4 × 3 × factorial(2) = … = 4 × 3 × 2 × 1 × 1 = 24.

Doğrudan ve dolaylı özyineleme (Slayt 12)

Doğrudan özyinelemede fonksiyon kendini çağırır. **Dolaylı** özyinelemede A, B'yi; B de A'yı çağırır.

PYTHON

```python
def cift_mi(n):
    if n == 0:
        return True
    return tek_mi(n - 1)

def tek_mi(n):
    if n == 0:
        return False
    return cift_mi(n - 1)
```

PYTHON (DEVAM)

```
print(cift_mi(4), tek_mi(7), cift_mi(3))
```

ÇIKTI

```
True True False
```

Liste aralığını toplamak (Slayt 13–14)

Temel durum: başlangıç indeksi bitişi geçtiyse 0. Özyinelemeli durum: o anki eleman + kalan aralığın toplamı.

PYTHON

```
def range_sum(num_list, start, end):
    if start > end:
        return 0
    else:
        return num_list[start] + range_sum(num_list, start + 1, end)

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(range_sum(numbers, 2, 5))
```

ÇIKTI

```
18
```

Fibonacci ve EBOB (Slayt 15–16)

Fibonacci serisinin **iki** temel durumu vardır: Fib(0) = 0, Fib(1) = 1; n > 1 için Fib(n) = Fib(n−1) + Fib(n−2).

EBOB (GCD): x, y'ye tam bölünüyorsa gcd(x, y) = y; değilse gcd(x, y) = gcd(y, x % y).

PYTHON

```
def fib(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)

def gcd(x, y):
    if x % y == 0:
        return y
    else:
        return gcd(y, x % y)
```

PYTHON (DEVAM)

```
print([fib(i) for i in range(10)])
print(gcd(49, 28), gcd(12, 18))
```

ÇIKTI

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
7 6
```

Hanoi kuleleri (Slayt 17–21)

Üç çubuk ve küçülen diskler. Kurallar: aynı anda **yalnızca bir disk** taşınır, büyük disk küçüğün üstüne konamaz, taşınan disk dışında tüm diskler bir çubukta durur. n diski 1'den 3'e taşımak için:

1. n − 1 diski 1'den 2'ye taşı (3'ü geçici kullanarak),
2. kalan en büyük diski 1'den 3'e taşı,
3. n − 1 diski 2'den 3'e taşı (1'i geçici kullanarak).

PYTHON

```
def move_discs(num, from_peg, to_peg, temp_peg):
    if num > 0:
        move_discs(num - 1, from_peg, temp_peg, to_peg)
        print('Move a disc from peg', from_peg, 'to peg', to_peg)
        move_discs(num - 1, temp_peg, to_peg, from_peg)

move_discs(3, 1, 3, 2)
```

ÇIKTI

```
Move a disc from peg 1 to peg 3
Move a disc from peg 1 to peg 2
Move a disc from peg 3 to peg 2
Move a disc from peg 1 to peg 3
Move a disc from peg 2 to peg 1
Move a disc from peg 2 to peg 3
Move a disc from peg 1 to peg 3
```

Üç disk **7** hamlede taşınır (genel olarak $2^n - 1$ hamle).

Özyineleme mi, döngü mü? (Slayt 22–23)

Özyineleme kullanmamanın nedenleri: fonksiyon çağrısı yükü yüzünden daha az verimlidir ve döngülü çözüm çoğu zaman daha anlaşılırdır. Ama tanımı zaten özyinelemeli olan bazı problemler (Fibonacci, Hanoi) özyinelemeyle daha doğal yazılır.

PYTHON
```python
def factorial_dongu(n):
    sonuc = 1
    for i in range(1, n + 1):
        sonuc *= i
    return sonuc

print(factorial_dongu(4), factorial_dongu(10))
```

ÇIKTI
```
24 3628800
```

Sık yapılan hatalar

- **Temel durumu unutmak ya da ona hiç ulaşamamak:** fonksiyon kendini sonsuza kadar çağırır, Python `RecursionError` ile durdurur.

PYTHON
```python
def geri_say(n):
    return geri_say(n - 1)

geri_say(5)
```

ÇIKTI
```
RecursionError: maximum recursion depth exceeded
```

- **Problemi küçültmemek:** özyinelemeli çağrı parametreyi temel duruma doğru değiştirmelidir (`n - 1` gibi).
- **Fibonacci'nin ikinci temel durumunu unutmak:** `fib(1)` için ayrı bir dönüş olmazsa `fib(-1)`'e iner.
- `return`**'ü unutmak:** `num * factorial(num - 1)` döndürülmezse sonuç `None` olur.

Kısa tekrar soruları

1. Temel durum ve özyinelemeli durum nedir?
2. `factorial(3)` çağrısında özyineleme derinliği kaçtır (temel durum dahil kaç çağrı yapılır)?
3. `range_sum([5, 1, 7, 3], 1, 3)` nedir?
4. `gcd(18, 12)` adım adım nasıl hesaplanır?
5. 4 diskli Hanoi kulesi kaç hamlede çözülür?
6. Özyinelemeli çözümlerin döngülü çözümlere göre bir dezavantajı nedir?

Cevaplar

1. Temel durum, özyineleme olmadan doğrudan çözülen durumdur; özyinelemeli durum, problemi daha küçük bir kopyasına indirgeyip fonksiyonu yeniden çağırır.
2. 4 çağrı: `factorial(3)`, `(2)`, `(1)`, `(0)`.
3. 1 + 7 + 3 = 11.
4. 18 % 12 = 6 ≠ 0 → gcd(12, 6); 12 % 6 = 0 → 6.
5. $2^4 - 1 = 15$ hamle.
6. Her fonksiyon çağrısının ek yükü olduğu için genellikle daha az verimlidir (ve çoğu zaman daha zor anlaşılır).